

UNIVERSIDAD ABIERTA PARA ADULTOS
ESCUELA DE INGENIERÍA Y TECNOLOGÍA

PROGRAMACIÓN DE DISPOSITIVOS MÓVILES
ISW-307

PROYECTO PRÁCTICO FINAL

Consigna, Rúbrica y Ejemplo de Aplicación

Ponderación	40 puntos (50% de la calificación final)
Modalidad	Equipo (4-5 integrantes) + Individual
Distribución	Documento técnico (12) + Código (12) + Presentación individual (16)
Fecha de entrega	Semana 11 — Defensa presencial
Stack	Ionic + Angular + Capacitor
Facilitador	Joan Manuel Gregorio Pérez
Periodo	2026-32 NRC: 1185

1. CONSIGNA DEL PROYECTO PRÁCTICO FINAL

Saludo, distinguidos participantes:

El proyecto práctico final es el entregable más importante de esta asignatura. Consiste en desarrollar una aplicación móvil completa utilizando Ionic + Angular + Capacitor que integre TODOS los temas vistos en las 10 unidades del curso.

El trabajo se realiza en equipos de 4 a 5 integrantes. Cada integrante es responsable de un módulo específico de la aplicación, que deberá defender individualmente en la semana 11.

La aplicación debe ser funcional, ejecutarse en un navegador y preferiblemente en un dispositivo móvil real.

2. DISTRIBUCIÓN DE LA CALIFICACIÓN (40 PUNTOS)

Componente	Puntos	Tipo	Descripción
A. Documento técnico	12 pts	Grupal	Informe del proyecto: arquitectura, diseño, desarrollo, pruebas
B. Código y funcionalidad	12 pts	Grupal	App Ionic funcionando con todos los módulos integrados
C. Presentación individual	16 pts	Individual	Cada integrante expone y defiende su módulo (4 pts × 4 criterios)
TOTAL	40 pts		50% de la calificación final

3. REQUERIMIENTOS FUNCIONALES POR UNIDAD

La aplicación DEBE incluir funcionalidades que cubran TODAS las unidades del curso:

Unidad	Tema	Requerimiento mínimo	Peso
1-2	Intro + Navegación	Estructura base con tabs, routing, lazy loading, mínimo 4 páginas	Obligatorio
3	Interfaces y gestos	Componentes Ionic (cards, lists, inputs) + gestos (swipe, pull-to-refresh)	Obligatorio
4	Conectividad	Detección de estado de red (online/offline) o WiFi	Obligatorio
5	Bluetooth/NFC	Integración BLE para escaneo de dispositivos O NFC para compartir datos	Obligatorio
6	Geolocalización	GPS + mapa interactivo con marcadores (Leaflet o Google Maps)	Obligatorio

7	Multimedia	Reproductor de audio O video con controles personalizados	Obligatorio
8	Cámara	Captura de foto O escaneo de código QR con @capacitor/camera	Obligatorio
9	Almacenamiento	CRUD con SQLite, IndexedDB o @ionic/storage	Obligatorio
10	Servicios Web	Consumo de API REST (GET, POST) con HttpClient o fetch	Obligatorio

4. EJEMPLO: CAMPUSAPP — GESTIÓN ACADÉMICA UAPA

A continuación se presenta un ejemplo de aplicación que cumple con TODOS los requerimientos del curso. Este es solo un ejemplo; los equipos pueden elegir cualquier tema que cubra las unidades.

4.1 Descripción general

CampusApp es una aplicación móvil para estudiantes de la UAPA que permite gestionar su vida académica: ver horarios, tomar notas, escanear asistencia, explorar el campus, escuchar podcasts educativos y consumir noticias académicas.

4.2 Módulos por integrante (equipo de 5)

#	Integrante	Módulo asignado	Unidades	Funcionalidades
1	Estudiante A	Navegación + Noticias	U1, U2, U10	Estructura base, tabs, routing, consumo de API REST para noticias
2	Estudiante B	Tareas + Interfaz	U3, U9	Lista de tareas con gestos (swipe delete, pull refresh), CRUD con SQLite
3	Estudiante C	Conectividad + Mapa	U4, U6	Detección de red online/offline, GPS con mapa Leaflet, marcadores
4	Estudiante D	Multimedia + Cámara	U7, U8	Reproductor de podcasts (audio), escanear QR de asistencia
5	Estudiante E	Bluetooth/NFC + Perfil	U5, U9	Compartir notas por NFC/BLE, perfil de usuario con almacenamiento local

4.3 Funcionalidades detalladas por módulo

Módulo 1: Navegación + Noticias (Estudiante A)

Unidades: 1 (Intro), 2 (Navegación), 10 (Servicios Web)

Componentes Ionic:

- ion-tabs con 5 tabs: Inicio, Tareas, Campus, Podcasts, Configuración
- ion-router-outlet con lazy loading
- ion-card para mostrar noticias
- ion-refresher para pull-to-refresh

Capacitor plugins:

- @angular/common/http para consumir API REST

- Interfaz con loading spinner y manejo de errores

API consumida:

- GET <https://jsonplaceholder.typicode.com/posts> (simula noticias)
- Muestra título, resumen y fecha de cada noticia

Módulo 2: Tareas + Interfaz (Estudiante B)

Unidades: 3 (Interfaces), 9 (Almacenamiento)

Componentes Ionic:

- ion-list con ion-item-sliding (swipe para eliminar)
- ion-input + ion-button para agregar tareas
- ion-checkbox para marcar como completada
- ion-reorder para reordenar tareas

Capacitor plugins:

- @capacitor-community/sqlite o @ionic/storage
- CRUD completo: Crear, Leer, Actualizar, Eliminar

Gestos implementados:

- Swipe izquierda: eliminar tarea
- Pull-to-refresh: recargar lista
- Long press: editar tarea

Módulo 3: Conectividad + Mapa (Estudiante C)

Unidades: 4 (Conectividad), 6 (Geolocalización)

Componentes Ionic:

- ion-badge para mostrar estado de red (online/offline)
- Mapa interactivo con Leaflet.js
- ion-fab para botón de ubicación actual

Capacitor plugins:

- @capacitor/geolocation para GPS
- @capacitor/network para detectar estado de red

Funcionalidades:

- Mostrar ubicación actual en el mapa
- Marcadores de edificios UAPA con popup informativo
- Indicador visual de conectividad (verde=online, rojo=offline)

Módulo 4: Multimedia + Cámara (Estudiante D)

Unidades: 7 (Multimedia), 8 (Cámara)

Componentes Ionic:

- Reproductor de audio con controles play/pause/stop
- ion-range para barra de progreso
- ion-list para lista de podcasts

Capacitor plugins:

- @capacitor/camera para captura de foto
- HTML5 Audio API para reproducción
- BarcodeDetector API o @capacitor-mlkit/barcode-scanning para QR

Funcionalidades:

- Lista de podcasts educativos con reproductor integrado
- Escanear QR para registrar asistencia
- Guardar foto de notas con la cámara

Módulo 5: Bluetooth/NFC + Perfil (Estudiante E)

Unidades: 5 (Bluetooth/NFC), 9 (Almacenamiento)

Componentes Ionic:

- ion-card para perfil de usuario
- ion-toggle para configuraciones
- ion-list para historial de compartidos

Capacitor plugins:

- @capacitor-community/bluetooth-le para BLE
- @capacitor-community/nfc para NFC
- @ionic/storage para perfil y configuraciones

Funcionalidades:

- Compartir notas/texto por NFC (tap-to-share)
- Escanear dispositivos BLE cercanos
- Perfil de usuario con foto, nombre, carrera

5. DOCUMENTO TÉCNICO — ESTRUCTURA DETALLADA (12 PUNTOS)

A continuación se describe QUÉ debe incluir cada sección del documento técnico y CÓMO redactarla:

5.1 Portada + Índice (0.5 puntos)

QUÉ es:

La primera impresión del documento. Debe verse profesional y contener toda la información necesaria para identificar el trabajo.

QUÉ debe incluir:

- Logo oficial de la UAPA (descargable de uapa.edu.do)
- Título del proyecto (nombre de la aplicación)
- Nombre de la asignatura: Programación de Dispositivos Móviles — ISW-307
- Nombres completos y matrículas de TODOS los integrantes del equipo
- Nombre del facilitador: Joan Manuel Gregorio Pérez
- Fecha de entrega
- Índice con números de página (generado automáticamente en Word)

EJEMPLO:

CampusApp: Sistema de Gestión Académica para Estudiantes UAPA — Desarrollado con Ionic + Angular + Capacitor — Equipo: Pérez J., Rodríguez M., García L., Díaz A., Santos C.

5.2 Introducción y Objetivos (1.0 puntos)

QUÉ es:

Una explicación clara del PROBLEMA que resuelve la aplicación y QUÉ se quiere lograr. Es la justificación del proyecto.

QUÉ debe incluir:

- Planteamiento del problema: ¿Qué necesidad o problema tiene el usuario? ¿Por qué esta app es necesaria?
- Objetivo general: Una oración que resuma qué hace la app (ej: 'Desarrollar una aplicación móvil que permita a los estudiantes UAPA gestionar su vida académica')
- Objetivos específicos: 3-5 objetivos medibles y concretos (ej: 'Implementar un módulo de tareas con persistencia local usando SQLite')
- Alcance: Qué incluye y qué NO incluye la app (ej: 'Incluye gestión de tareas y mapa del campus. No incluye sistema de calificaciones')
- Público objetivo: ¿Para quién es la app? (ej: 'Estudiantes de pregrado de la UAPA')

EJEMPLO de objetivo SMART:

Objetivo específico 1: Desarrollar un módulo de lista de tareas con operaciones CRUD completas (Crear, Leer, Actualizar, Eliminar) utilizando @ionic/storage, que permita al estudiante organizar sus pendientes académicos con persistencia de datos local.

5.3 Modelo de Negocio — Canvas (1.0 puntos)

QUÉ es:

El Lienzo Canvas (Business Model Canvas) es una herramienta visual que describe cómo la aplicación crea, entrega y captura valor. Define los 9 bloques fundamentales del negocio.

QUÉ debe incluir:

- Segmento de clientes: ¿A quién va dirigida la app? (estudiantes, profesores, administración)
- Propuesta de valor: ¿Qué problema resuelve y por qué el usuario la elegiría?
- Canales: ¿Cómo llega la app al usuario? (Google Play, App Store, web)
- Relación con el cliente: ¿Cómo interactúa con el usuario? (soporte, feedback, updates)
- Fuentes de ingreso: ¿Cómo genera dinero? (gratis, freemium, ads, suscripción)
- Recursos clave: ¿Qué necesita para funcionar? (servidores, APIs, equipo de desarrollo)
- Actividades clave: ¿Qué hace la empresa? (desarrollo, mantenimiento, soporte)
- Socios clave: ¿Quiénes son los aliados? (UAPA, proveedores de APIs, Google/Apple)
- Estructura de costos: ¿Cuánto cuesta desarrollar y mantener la app?

HERRAMIENTA sugerida: Canva tiene plantillas de Canvas gratuitas. También puede usar Strategyzer.com.

5.4 Arquitectura Técnica (1.5 puntos)

QUÉ es:

Un diagrama y descripción de cómo está construida la aplicación por dentro. Muestra las capas, componentes y cómo se comunican entre sí.

QUÉ debe incluir:

- Diagrama de arquitectura: Dibujar con Draw.io o Lucidchart mostrando las capas de la app (Vista → Componente → Servicio → API/Storage)
- Stack tecnológico: Listar todas las tecnologías usadas (Ionic 7, Angular 17, Capacitor 5, TypeScript, etc.)
- Plugins Capacitor: Tabla con cada plugin instalado, su versión y para qué se usa
- Estructura de carpetas: Captura del árbol de archivos del proyecto (src/app/pages, services, models, etc.)
- Flujo de datos: Cómo fluyen los datos desde la API hasta la pantalla del usuario

EJEMPLO de tabla de plugins:

Plugin	Versión	Uso en el proyecto
@capacitor/geolocation	5.0.0	Obtener ubicación GPS del estudiante para el mapa del campus
@capacitor/camera	5.0.0	Capturar fotos de notas y escanear códigos QR de asistencia
@capacitor-community/sqlite	5.0.0	Almacenar tareas y notas localmente con persistencia
@capacitor/network	5.0.0	Detectar si el dispositivo tiene conexión a internet

5.5 Diseño de Interfaces (1.5 puntos)

QUÉ es:

El diseño visual de la aplicación antes de programarla. Muestra cómo se verá cada pantalla y la experiencia del usuario.

QUÉ debe incluir:

- Wireframes: Bocetos de cada pantalla de la app (puede usar Figma, Draw.io o papel fotografiado)
- Paleta de colores: Definir 3-5 colores principales con sus códigos hexadecimales (ej: primario #6366F1, secundario #06B6D4)
- Guía de estilo: Tipografía usada (Inter, Roboto), tamaños de fuente, espaciado, bordes redondeados
- Mapa de navegación: Diagrama que muestra cómo el usuario se mueve entre pantallas
- Componentes reutilizables: Lista de componentes que se repiten (cards, botones, headers)

EJEMPLO de paleta de colores:

Primario: #6366F1 (violeta) — Botones principales, tab activo | Secundario: #06B6D4 (cyan) — Enlaces, iconos | Éxito: #10B981 (verde) — Confirmaciones | Peligro: #EF4444 (rojo) — Eliminar, errores | Fondo: #F1F5F9 (gris claro) — Fondo de pantallas

5.6 Desarrollo por Módulos (3.0 puntos)

QUÉ es:

La sección más extensa del documento. Cada integrante del equipo documenta SU módulo explicando qué problema resolvió, cómo lo hizo y qué código escribió.

QUÉ debe incluir por cada módulo (1 subsección por integrante):

- Nombre del módulo y responsable: Ej: 'Módulo de Tareas — María Rodríguez'
- Problema a resolver: ¿Qué necesidad del usuario atiende este módulo?
- Descripción de la solución: ¿Cómo funciona? ¿Qué componentes Ionic usa?
- Código relevante: Fragmentos de código TypeScript/HTML comentados (no todo el código, solo lo más importante)

- Capturas de pantalla: Al menos 2-3 capturas del módulo funcionando en el navegador o dispositivo
- Decisiones técnicas: ¿Por qué eligió SQLite y no localStorage? ¿Por qué usó ion-item-sliding?
- Problemas encontrados y soluciones: ¿Qué errores tuvo? ¿Cómo los resolvió?

EJEMPLO de documentación de módulo:

Módulo: Lista de Tareas con Gestos — Responsable: María Rodríguez — Problema: Los estudiantes necesitan organizar sus pendientes académicos de forma rápida desde su celular. — Solución: Se implementó una lista de tareas usando ion-list con ion-item-sliding que permite eliminar tareas deslizando a la izquierda. Los datos se almacenan con @ionic/storage para que persistan al cerrar la app. — Código: [fragmento del servicio storage.service.ts con comentarios] — Capturas: [pantalla de la lista, swipe para eliminar, formulario de agregar]

5.7 Pruebas y Despliegue (1.0 puntos)

QUÉ es:

Evidencia de que la aplicación funciona correctamente. Demuestra que se probó en diferentes escenarios y dispositivos.

QUÉ debe incluir:

- Tabla de pruebas: Lista de pruebas realizadas con resultado (pasó/falló)
- Capturas de la app ejecutándose: En el navegador (ionic serve), en emulador Android, en dispositivo real
- Pruebas de cada módulo: Verificar que cada funcionalidad funciona (agregar tarea, ver mapa, escanear QR, etc.)
- Pruebas de navegación: Verificar que todas las pantallas se conectan correctamente
- Bugs encontrados: Lista de errores encontrados y cómo se resolvieron (o si quedaron pendientes)
- Instrucciones de instalación: Pasos para clonar el repositorio e instalar la app (npm install, ionic serve)

EJEMPLO de tabla de pruebas:

#	Prueba realizada	Resultado	Observaciones
1	Agregar una nueva tarea	✅ Pasó	La tarea se guarda en storage y aparece en la lista
2	Eliminar tarea con swipe	✅ Pasó	El swipe a la izquierda muestra el botón de eliminar
3	Ver mapa con ubicación actual	✅ Pasó	El GPS marca la posición correcta en el campus
4	Escanear código QR	⚠️ Parcial	Funciona en Chrome pero falla en Firefox (API no soportada)
5	Consumir API de noticias	✅ Pasó	Las noticias se cargan con

			loading spinner
--	--	--	-----------------

5.8 Conclusiones (1.0 puntos)

QUÉ es:

Una reflexión final sobre lo aprendido, los logros alcanzados y las recomendaciones para el futuro.

QUÉ debe incluir:

- Logros del proyecto: ¿Qué se logró cumplir? ¿La app cumple con los requerimientos?
- Lecciones aprendidas: ¿Qué aprendió el equipo sobre desarrollo móvil con Ionic?
- Dificultades encontradas: ¿Qué fue lo más difícil? ¿Qué harían diferente?
- Recomendaciones: ¿Qué mejoras se podrían hacer en el futuro? ¿Qué funcionalidades faltan?
- Trabajo futuro: ¿Qué se implementaría en una versión 2.0 de la app?

EJEMPLO:

Como conclusión, el equipo logró desarrollar una aplicación funcional que integra las 10 unidades del curso. El mayor desafío fue la integración del módulo de Bluetooth Low Energy, ya que requirió un dispositivo físico para las pruebas. Recomendamos que futuros equipos comiencen con la arquitectura antes de programar y que utilicen el emulador de Android Studio para pruebas de geolocalización.

5.9 Referencias APA (0.5 puntos)

QUÉ es:

La lista de todas las fuentes consultadas durante el desarrollo del proyecto, en formato APA 7.ª edición.

QUÉ debe incluir:

- Mínimo 10 fuentes
- Al menos 3 fuentes de los últimos 5 años (2021-2026)
- Formato APA 7.ª edición con URL o DOI
- Tipos: documentación oficial, libros, artículos, tutoriales, Stack Overflow

EJEMPLOS de formato APA:

- Ionic Framework. (2025). Ionic Documentation. <https://ionicframework.com/docs>
- Sanz, R. L. y Galán López, R. (2014). Introducción a la movilidad. Dextra Editorial.
- Angular. (2025). Angular Documentation. <https://angular.dev>

5.10 Anexos (1.0 puntos)

QUÉ es:

Material adicional que complementa el documento principal pero es demasiado extenso para incluirlo en el cuerpo.

QUÉ debe incluir:

- Diagramas adicionales: Diagrama de clases, diagrama de secuencia, diagrama de base de datos
- Código fuente completo: Los archivos más importantes del proyecto (app.module.ts, servicios clave, modelos)
- Capturas adicionales: Pantallas de la app que no se mostraron en el cuerpo del documento
- Manual de usuario: Instrucciones básicas de cómo usar la app
- Actas de reuniones del equipo: Fechas, acuerdos, distribución de tareas

NOTA: Los anexos se numeran como Anexo A, Anexo B, Anexo C, etc. Cada anexo debe referenciarse en el cuerpo del documento.

6. RÚBRICA DE EVALUACIÓN (40 PUNTOS)

A. Documento Técnico (12 puntos)

Criterio	Pts	Excelente	Bueno	Aceptable	Insuficiente
1. Portada + índice	0.5	Completo, profesional, con logo UAPA	Presente, algunos datos faltantes	Básico	Ausente
2. Intro + objetivos	1.0	Problema claro, objetivos SMART, alcance definido	Objetivos presentes	Vagos	Sin objetivos
3. Canvas	1.0	9 bloques completos con análisis	7-8 bloques	4-6 bloques	Sin Canvas
4. Arquitectura	1.5	Diagrama claro, stack documentado, plugins listados	Diagrama básico	Descripción textual	Sin arquitectura
5. Diseño interfaces	1.5	Wireframes profesionales, paleta, guía de estilo	Wireframes presentes	Bocetos básicos	Sin diseño
6. Desarrollo módulos	3.0	Cada módulo documentado con código, explicación y capturas	Documentación presente	Descripción básica	Sin documentar
7. Pruebas	1.0	Capturas reales, pruebas en dispositivo, bugs documentados	Capturas presentes	Pocas pruebas	Sin pruebas
8. Conclusiones	1.0	Reflexión profunda, recomendaciones, trabajo futuro	Conclusiones básicas	Genéricas	Ausentes
9. Referencias	0.5	10+ fuentes APA correcto	8-9 fuentes	5-7 fuentes	< 5 fuentes
10. Anexos	1.0	Diagramas, código, capturas adicionales	Anexos básicos	Pocos	Sin anexos

B. Código y Funcionalidad (12 puntos)

Criterio	Pts	Excelente	Bueno	Aceptable	Insuficiente
1. Estructura proyecto	2.0	Arquitectura limpia, componentes organizados, lazy loading	Estructura correcta	Algo desorganizado	Sin estructura
2. Interfaz usuario	2.0	UI profesional, responsive,	UI funcional y atractiva	Básica pero funcional	Rota o sin estilo

		iconos, animaciones, tema consistente			
3. Navegación	1.5	Tabs + páginas + back button + transiciones	Navegación funcional	Básica	Sin navegación
4. Conectividad	1.5	WiFi/BT/NFC con plugin real, manejo de estados	Al menos 1 tecnología	Intento presente	Sin conectividad
5. Geolocalización	1.5	GPS + mapa interactivo + marcadores + geofencing	GPS + mapa básico	Solo coordenadas	Sin GPS
6. Multimedia	1.0	Audio/video con controles personalizados, lista de medios	Reproductor funcional	Básico	Sin multimedia
7. Almacenamiento	1.0	CRUD completo con SQLite, persistencia offline	Lectura/escritura	Solo localStorage	Sin persistencia
8. Servicios web	1.5	API REST: GET + POST, loading, errores, caché	GET funcional con loading	GET básico	Sin consumo API

C. Presentación Individual (16 puntos)

Cada integrante del equipo presenta SU módulo individualmente ante el facilitador.

Tiempo: 5 minutos de exposición + 2-3 preguntas del facilitador.

Criterio	Pts	Excelente	Bueno	Aceptable	Insuficiente
1. Dominio del tema	4	Explica con profundidad, terminología técnica, comprende todo su módulo	Explica bien, alguna imprecisión	Explica superficialmente, lee notas	No demuestra comprensión
2. Demo en vivo	4	App funciona sin errores, muestra todas las funciones de su módulo	Funciona con errores menores	Funciona parcialmente	No funciona o no puede demostrar
3. Calidad exposición	4	Claro, estructurado, dentro del tiempo, buena presencia, recursos visuales	Buena presentación, algunos tropiezos	Improvisada, desordenada	No se entiende o fuera de tiempo

4. Respuesta preguntas	4	Responde todas con precisión y ejemplos técnicos	Responde la mayoría correctamente	Responde parcialmente	No responde o responde mal
-------------------------------	---	--	-----------------------------------	-----------------------	----------------------------

NOTA FINAL = Promedio individual (16 pts) + Nota grupal (24 pts) = 40 puntos

7. CRONOGRAMA DE DESARROLLO

Semana	Actividad	Entregable
1-2	Formar equipos (4-5), elegir tema, aprobar con facilitador	Tema aprobado
3	Diseñar arquitectura, wireframes, modelo Canvas	Diagrama + wireframes + Canvas
4-5	Desarrollo módulo 1-2 (navegación + interfaces/tareas)	Avance 1: app base + lista tareas
6-7	Desarrollo módulo 3-4 (conectividad + GPS + multimedia + cámara)	Avance 2: mapa + reproductor + QR
8-9	Desarrollo módulo 5 (BLE/NFC + almacenamiento + API)	Avance 3: app completa funcional
10	Integración, pruebas, documento técnico	App integrada + borrador documento
11	DEFENSA INDIVIDUAL + Entrega final	Documento + código + presentación

8. ESTRUCTURA DEL REPOSITORIO

El código fuente debe subirse a GitHub con la siguiente estructura:

campus-app/

- `src/app/pages/` — Páginas de la app (home, tareas, campus, podcasts, config)
- `src/app/services/` — Servicios (api.service, storage.service, geolocation.service)
- `src/app/models/` — Interfaces TypeScript (Tarea, Noticia, Podcast, etc.)
- `src/app/components/` — Componentes reutilizables (card-noticia, reproductor, etc.)
- `src/assets/` — Imágenes, iconos, archivos de audio
- `README.md` — Descripción, instrucciones de instalación, capturas
- `package.json` — Dependencias del proyecto

README.md obligatorio:

- Nombre de la app y descripción breve (2-3 oraciones)
- Tecnologías utilizadas (Ionic, Angular, Capacitor, plugins)
- Instrucciones de instalación: `git clone` → `npm install` → `ionic serve`
- Capturas de pantalla de cada módulo
- Equipo: nombres, matrículas, módulos asignados

9. CHECKLIST PRE-DEFENSA

- [] ¿App compila y ejecuta sin errores (ionic serve)?
- [] ¿Todas las pantallas funcionan correctamente?
- [] ¿Navegación entre páginas y tabs operativa?
- [] ¿Gestos implementados (swipe, pull-to-refresh)?
- [] ¿Conectividad detectada (online/offline)?
- [] ¿GPS y mapa funcionando con marcadores?
- [] ¿Multimedia reproduciendo (audio o video)?
- [] ¿Cámara o QR escaneando correctamente?
- [] ¿Almacenamiento persistiendo datos (SQLite o similar)?
- [] ¿API REST consumida y mostrando datos?
- [] ¿BLE o NFC implementado (al menos básico)?
- [] ¿Documento técnico completo (25+ páginas)?
- [] ¿README.md con instrucciones y capturas?
- [] ¿Código subido a GitHub?
- [] ¿Diapositivas de presentación preparadas (por integrante)?
- [] ¿Cada integrante conoce su módulo y puede defenderlo?

10. POLÍTICA DE INTEGRIDAD

El código debe ser desarrollado por los miembros del equipo.

✓ PERMITIDO:

- Usar documentación oficial de Ionic, Angular, Capacitor
- Adaptar ejemplos de la documentación oficial
- Usar Stack Overflow para resolver errores específicos
- Usar IA para entender conceptos o depurar errores

✗ NO PERMITIDO:

- Copiar código completo de repositorios de GitHub sin adaptar
- Pedir a IA que genere módulos completos
- Usar plantillas pre-hechas sin modificar significativamente
- Comprar o descargar proyectos terminados

El facilitador puede hacer preguntas técnicas durante la defensa para verificar que el estudiante comprende el código que presentó.

11. RECURSOS DE APOYO

Recurso	URL	Para qué sirve
Ionic Documentation	ionicframework.com/docs	Componentes, plugins, CLI de Ionic

Angular Documentation	angular.dev	Componentes, servicios, routing, forms
Capacitor Documentation	capacitorjs.com/docs	Plugins nativos: cámara, GPS, BLE, NFC
Leaflet.js	leafletjs.com	Mapas interactivos gratuitos
SQLite Plugin	github.com/capacitor-community/sqlite	Base de datos local SQLite
BLE Plugin	github.com/capacitor-community/bluetooth-le	Bluetooth Low Energy
NFC Plugin	github.com/capacitor-community/nfc	Near Field Communication
Barcode Scanner	github.com/capacitor-community/mlkit/barcode-scanning	Escanear códigos QR
Draw.io	app.diagrams.net	Diagramas de arquitectura y wireframes
Figma	figma.com	Diseño de interfaces y prototipos